Aprendizagem de Máquina

Aula 06 — Classificação

Prof. Dr. Jonas Krause
PUCPR · PPGCIS · Ciência da Computação

10 de setembro de 2026

A Aula 06 segue diretamente o cronograma do plano de ensino

Semanas 5–7

Regressão e classificação. Implementação de modelos supervisionados.

Aula 05

Regressão: previsão de valores contínuos.

Aula 06

Classificação: previsão de categorias.

A regressão prepara a passagem para a classificação



Continuidade

O fluxo supervisionado permanece; mudam o tipo de target, a saída e as métricas.

Ao final, o estudante deverá implementar uma classificação completa

1. reconhecer problemas de classificação;
2. diferenciar quantidades e categorias;
3. identificar features e target;
4. separar treino e teste;
5. criar um baseline;
6. treinar regressão logística;
7. produzir classes previstas;
8. construir a matriz de confusão;
9. interpretar métricas básicas;
10. executar o fluxo em Python.

A aula acompanha a ordem real de construção do modelo

Parte I — Compreender e treinar

1. reconhecer a tarefa;
2. preparar treino e teste;
3. estabelecer um baseline;
4. treinar o classificador.

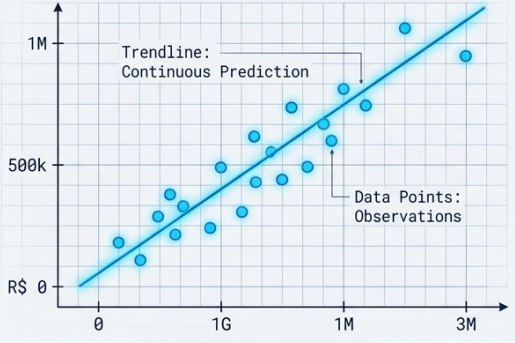
Parte II — Avaliar e interpretar

5. construir a matriz de confusão;
6. calcular métricas;
7. analisar erros;
8. comunicar a conclusão.

Regressão vs. Classificação

Da previsão de quantidades contínuas para a separação de categorias.

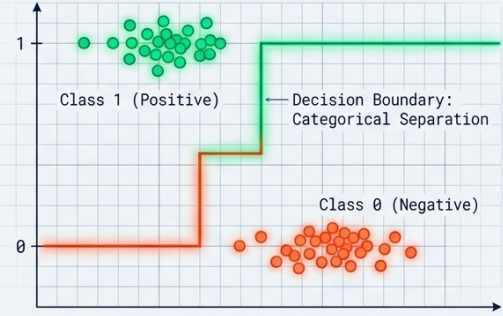
Quantidades



Quanto?

(Valor Numérico)

Categorias



O Quê?

(Categoria Discreta)

Gemini Notebook

Classificação prevê categorias conhecidas

Definição

Classificação é uma tarefa supervisionada na qual o modelo aprende, a partir de exemplos rotulados, a atribuir uma categoria a uma nova observação.

No treinamento

- recebe features;
- recebe a classe correta;
- aprende padrões entre elas.

Na utilização

- recebe apenas as features;
- aplica o padrão aprendido;
- retorna uma classe prevista.

O significado do target determina o tipo de problema

Pergunta	Tarefa	Target	
Qual será a temperatura?	Regressão	graus Celsius	
O tumor é maligno?	Classificação	sim / não	
Qual será o consumo?	Regressão	kWh	**Quantidade**
Qual é a espécie da flor?	Classificação	três espécies	
O cliente cancelará?	Classificação	sim / não	

a estimar: regressão. Categoria a escolher: classificação.

Um número também pode representar uma categoria

Número quantitativo

Temperatura de 20, 21 ou 22 °C. A distância entre valores possui significado.

Tarefa: regressão.

Código numérico

Classes 0, 1 e 2 usadas como rótulos de espécies. A distância não representa quantidade.

Tarefa: classificação.

Cada linha possui features e uma classe correta

raio médio	textura média	perímetro médio	diagnóstico
17,99	10,38	122,80	maligno
12,45	15,70	82,57	benigno
16,13	20,68	108,10	maligno

Features
Medidas usadas para produzir a previsão.

Target
Classe que o modelo deverá aprender a prever.

A unidade de análise define o que o modelo classifica

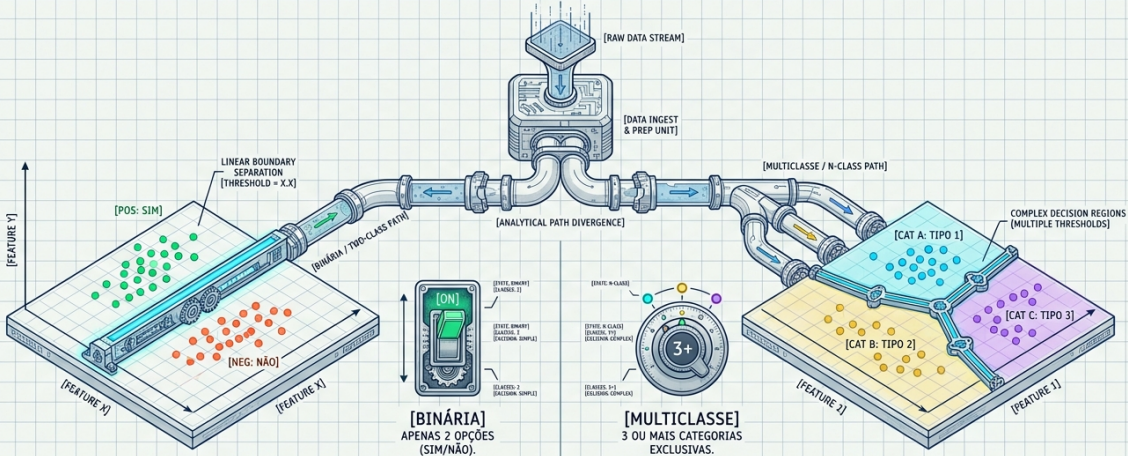
1. O que representa cada linha: paciente, flor ou transação?
2. Qual categoria será prevista?
3. Em qual momento a previsão será feita?
4. Quais features estarão disponíveis nesse momento?
5. Como a previsão será usada?

Atenção

Um modelo pode estar matematicamente correto e responder à pergunta errada.

BINÁRIA VS. MULTICLASSE

O LIMITE DAS CATEGORIAS QUE O MODELO PRECISA APRENDER A SEPARAR.



[ENGINEERING NOTE: BINARY = ONE BOUNDARY, SIMPLE DECISION. MULTICLASS = MULTIPLE BOUNDARIES, COMPLEX REGION MAPPING.]

Gemini Notebook

A classificação pode ser binária ou multiclasse

Binária

Duas classes possíveis.

- benigno / maligno;
- fraude / legítima;
- cancela / permanece.

Multiclasse

Três ou mais classes possíveis.

- três espécies de flores;
- categorias de produtos;
- dígitos de 0 a 9.

O dataset deve ser separado em features e target

```
X = df.drop(columns="target") # dados de entrada
y = df["target"] # classe correta
```

- X contém as colunas usadas pelo modelo;
- y contém o rótulo que queremos prever;
- o target nunca deve permanecer dentro de X;
- cada linha de X corresponde à mesma linha de y.

Treino ensina o modelo; teste verifica a generalização

Treino

- usado pelo método `fit`;
- permite aprender parâmetros;
- contém a maior parte dos dados.

Teste

- permanece separado;
- simula observações novas;
- verifica a generalização.

O Papel da Estratificação

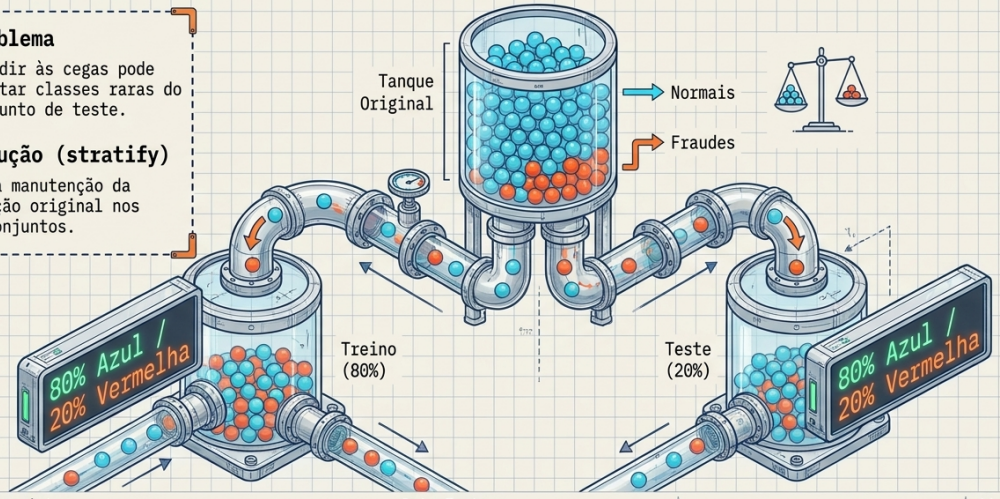
Mantendo a balança justa: proporções idênticas no Treino e no Teste.

O Problema

- Dividir às cegas pode ocultar classes raras do conjunto de teste.

A Solução (stratify)

Força a manutenção da proporção original nos dois conjuntos.



Gemini Notebook

A divisão treino–teste preserva as classes

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)
```

- 80% dos exemplos serão usados no treino;
- 20% serão reservados para o teste;
- `random_state` permite reproduzir a divisão;
- `stratify=y` preserva aproximadamente a proporção das classes.

Estratificar evita uma divisão acidentalmente desequilibrada

	Classe A	Classe B
Dataset completo	70%	30%
Treino estratificado	aprox. 70%	aprox. 30%
Teste estratificado	aprox. 70%	aprox. 30%

Por que isso importa?

Treino e teste precisam representar as classes para que o aprendizado e a avaliação façam sentido.

O baseline estabelece o desempenho mínimo esperado

Baseline

Um modelo propositalmente simples usado como referência. Ele pode prever sempre a classe mais frequente do treino.`DummyClassifier(strategy="most_frequent")`

Se o classificador vence
As features contêm informação útil.

Se não vence
Problema, dados ou modelo precisam ser revistos.

A regressão logística é usada para classificação

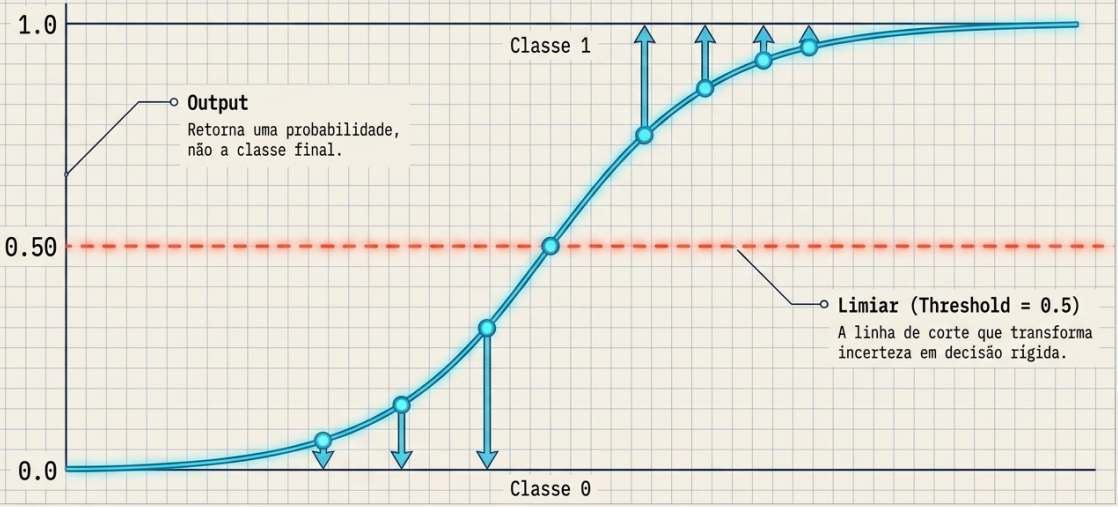
- apesar do nome, sua saída permite escolher uma classe;
- combina as features em um valor numérico;
- transforma esse valor em uma probabilidade entre 0 e 1;
- converte a probabilidade em classe;
- é um primeiro modelo simples para implementar classificação.

Foco da aula

Compreender e executar o fluxo supervisionado de classificação.

Regressão Logística & O Limiar de Decisão

Convertendo matemática em probabilidades contidas entre 0 e 1.

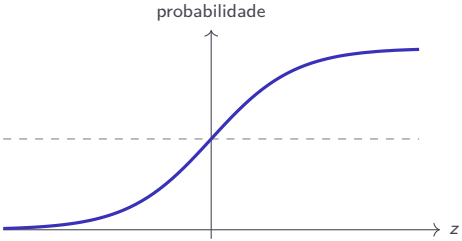


Gemini Notebook

A função logística transforma um valor em probabilidade

$$z = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

$$P(y = 1 \mid X) = \frac{1}{1 + e^{-z}}$$



A probabilidade é convertida em uma classe

$$P(y = 1 \mid X) = 0{,}82$$

Menor que 0,50

prever classe 0

Igual ou maior que 0,50

prever classe 1

O método `predict` já retorna a classe prevista.

O pipeline conecta padronização e classificação

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
model = Pipeline([
    ("scale", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=1000))
])
```

- o escalonamento é aprendido apenas com o treino;
- a mesma transformação é aplicada ao teste;
- o classificador recebe dados processados de forma consistente.

Treinar e prever exigem duas operações principais

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

1. `fit`: aprende o escalonamento e o classificador com o treino;
2. `predict`: aplica o pipeline aos exemplos de teste;
3. `y_pred`: guarda uma classe prevista para cada linha de `X_test`.

Não faça

Não execute `fit` usando o conjunto de teste.

Exercício 1 — classificação binária completa

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target
Xtr, Xte, ytr, yte = train_test_split(
 X, y, test_size=.2, random_state=42, stratify=y)
model = Pipeline([("scale", StandardScaler()),
 ("clf", LogisticRegression(max_iter=1000))])
model.fit(Xtr, ytr)
pred = model.predict(Xte)
print(X.shape, data.target_names)
print("Accuracy:", accuracy_score(yte, pred))
print(confusion_matrix(yte, pred))
```

Execute e responda

1. Quantas linhas e features existem?
2. O que significam 0 e 1?
3. Quantos exemplos ficaram no teste?
4. Qual foi a accuracy?
5. Quantos erros estão fora da diagonal?
6. O resultado supera a classe majoritária?

Avaliar significa comparar a classe real com a prevista

Exemplo	Classe real	Classe prevista
1	benigno	benigno
2	maligno	benigno
3	maligno	maligno
4	benigno	maligno

- exemplos 1 e 3 foram classificados corretamente;
- exemplo 2 é um maligno previsto como benigno;
- exemplo 4 é um benigno previsto como maligno;
- a matriz de confusão resume essas combinações.

A Matriz de Confusão

Dissecando os acertos absolutos e os dois tipos letais de erros.



Gemini Notebook

A matriz de confusão organiza acertos e erros

	Predição	
Realidade	Negativa	Positiva
Negativa	**TN**	**FP**
Positiva	**FN**	**TP**

Diagonal principal
TN e TP são previsões corretas.

Fora da diagonal
FP e FN são os dois tipos de erro.

As Métricas de Desempenho

Qual pergunta o seu modelo realmente precisa responder?



Acurácia

De todas as previsões, quantas eu acertei no total?



Precisão

Dos que eu avisei que eram Positivos, quantos realmente eram?

(Cuidado com Falso Alarme)



Sensibilidade (Recall)

De todos os Positivos reais que existiam, quantos eu consegui capturar?

(Não deixe passar nada)



F1-Score

Qual é o equilíbrio perfeito entre Precisão e Recall?

(Média harmônica)

Gemini Notebook

Os quatro resultados dependem da classe positiva

- **TP:** era positivo e foi previsto como positivo;
- **TN:** era negativo e foi previsto como negativo;
- **FP:** era negativo, mas foi previsto como positivo;
- **FN:** era positivo, mas foi previsto como negativo.

Antes de interpretar

Declare qual rótulo está sendo tratado como positivo.

Accuracy mede a proporção total de acertos

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- considera todas as previsões;
- pergunta qual proporção foi classificada corretamente;
- é fácil de comunicar;
- deve ser acompanhada da matriz de confusão.

Exemplo

90 acertos em 100 exemplos correspondem a 90% de accuracy.

Precision verifica a qualidade das previsões positivas

$$\text{Precision} = \frac{TP}{TP + FP}$$

- considera apenas os exemplos previstos como positivos;
- pergunta quantos positivos previstos eram realmente positivos;
- diminui quando existem muitos falsos positivos.

Exemplo

15 alertas corretos entre 20 alertas: precision $= 15/20 = 0{,}75$.

Recall verifica quantos positivos reais foram encontrados

$$\text{Recall} = \frac{TP}{TP + FN}$$

- considera todos os exemplos realmente positivos;
- pergunta quantos positivos reais foram detectados;
- diminui quando existem muitos falsos negativos.

Exemplo

16 positivos detectados entre 20 existentes: recall $= 16/20 = 0{,}80$.

F1 combina precision e recall

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- utiliza a média harmônica;
- só será alto quando precision e recall forem altos;
- resume o equilíbrio entre os dois tipos de erro;
- não substitui a matriz de confusão.

Cada métrica responde a uma pergunta diferente

Métrica	Pergunta	Erro destacado
Accuracy	Quantas previsões estão corretas?	FP e FN reduzem os acertos.
Precision	Positivos previstos são confiáveis?	FP reduz precision.
Recall	Positivos reais foram encontrados?	FN reduz recall.
F1	Precision e recall estão equilibrados?	Um valor baixo limita F1.

A

matriz explica o erro; as métricas resumem perspectivas.

O scikit-learn calcula as métricas diretamente

```
from sklearn.metrics import (accuracy_score, precision_score,
    recall_score, f1_score, confusion_matrix)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1:", f1_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

Na classificação multiclasse, cada categoria é avaliada

Exemplo: Iris

- setosa;
- versicolor;
- virginica.

A avaliação mostra

- desempenho por espécie;
- quais espécies são confundidas;
- uma média entre as classes.

Matriz multiclasse

Cada linha representa uma classe real e cada coluna uma classe prevista.

O classification report organiza as métricas por classe

```
from sklearn.metrics import classification_report
print(classification_report(
    y_test, y_pred,
    target_names=class_names,
    digits=3
))
```

- cada linha corresponde a uma classe;
- `support` informa quantos exemplos reais existem;
- a média resume o conjunto, mas não substitui as classes individuais.

Baseline e classificador devem usar a mesma divisão

```
from sklearn.dummy import DummyClassifier
baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(X_train, y_train)
pred_base = baseline.predict(X_test)
pred_model = model.predict(X_test)
print("Baseline:", accuracy_score(y_test, pred_base))
print("Modelo:", accuracy_score(y_test, pred_model))
```

Comparação justa

Mesmos exemplos de treino, teste e mesma métrica.

Analisar erros explica o que a média esconde

1. observe as células fora da diagonal;
2. identifique quais classes são confundidas;
3. examine exemplos incorretos;
4. verifique dados incompletos ou inconsistentes;
5. compare com o baseline;
6. descreva uma possível melhoria.

Conclusão

Informe o resultado, explique os erros e diga se o modelo atende ao objetivo.

Exercício 2 — classificação multiclasse completa

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
iris = load_iris(as_frame=True)
X, y = iris.data, iris.target
Xtr, Xte, ytr, yte = train_test_split(
 X, y, test_size=.2, random_state=42, stratify=y)
model = Pipeline([("scale", StandardScaler()),
 ("clf", LogisticRegression(max_iter=1000))])
model.fit(Xtr, ytr)
pred = model.predict(Xte)
print(confusion_matrix(yte, pred))
print(classification_report(yte, pred,
 target_names=iris.target_names, digits=3))
```

Execute e entregue

1. Dimensão de X e features.
2. Quantidade por classe.
3. Matriz de confusão identificada.
4. Classe com menor recall.
5. Accuracy calculada pela matriz.
6. Conclusão de cinco linhas.

O notebook deve registrar o fluxo completo

- ☐ carregamento do dataset;
- ☐ descrição de features e target;
- ☐ distribuição das classes;
- ☐ divisão treino–teste;
- ☐ pipeline;
- ☐ treinamento;
- ☐ previsões;
- ☐ baseline;
- ☐ matriz de confusão;
- ☐ métricas;
- ☐ análise dos erros;
- ☐ conclusão.

A Aula 06 implementa a etapa de classificação

- classificação prevê categorias usando exemplos rotulados;
- features e target são separados;
- treino e teste possuem funções diferentes;
- o baseline estabelece a referência;
- a regressão logística produz classes previstas;
- a matriz de confusão organiza acertos e erros;
- accuracy, precision, recall e F1 descrevem o desempenho;
- o pipeline permite executar o fluxo completo.

A próxima aula conclui os modelos supervisionados

Ainda nas Semanas 5–7

Regressão e classificação. Implementação de modelos supervisionados.

Aula 07 — consolidação

Comparar regressão e classificação, revisar a implementação e preparar a avaliação da Semana 8.**Árvores e métodos de conjunto permanecem para as Semanas 9–11.**

Referências do plano de ensino

- GÉRON, A. *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. 3. ed. O'Reilly, 2022.
- JAMES, G.; WITTEN, D.; HASTIE, T.; TIBSHIRANI, R. *An Introduction to Statistical Learning: with Applications in Python*. 2. ed. Springer, 2023.
- HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. *The Elements of Statistical Learning*. 2. ed. Springer, 2009.
- SCIKIT-LEARN. Documentação dos datasets, regressão logística, métricas e pipelines.

Perguntas?

Aprendizagem de Máquina — Aula 06